# Arithmetic Circuits

C1: Adder

- Full Adder
- Carry-Ripple Adder
- Carry-Select Adder
- Carry-Lookahead Adder

C2: Multiplier

- Array Multiplier with CPA
- Array Multiplier with CSA



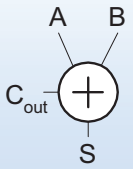


# Single-Bit Addition

## Half Adder

$S = A \oplus B$

$C_{out} = A \cdot B$

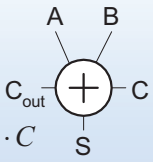



| A | B | $C_{out}$ | S |
|---|---|---|---|
| 0 | 0 | | |
| 0 | 1 | | |
| 1 | 0 | | |
| 1 | 1 | | |

## Full Adder

$S = A \oplus B \oplus C$

$C_{out} = A \cdot B + A \cdot C + B \cdot C$



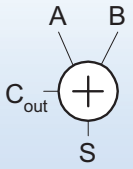

| A | B | C | $C_{out}$ | S |
|---|---|---|---|---|
| 0 | 0 | 0 | | |
| 0 | 0 | 1 | | |
| 0 | 1 | 0 | | |
| 0 | 1 | 1 | | |
| 1 | 0 | 0 | | |
| 1 | 0 | 1 | | |
| 1 | 1 | 0 | | |
| 1 | 1 | 1 | | |

# Single-Bit Addition

## Half Adder

$S = A \oplus B$

$C_{out} = A \cdot B$

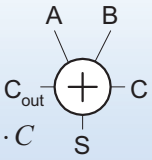



| A | B | $C_{out}$ | S |
|---|---|---|---|
| 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 0 | 1 |
| 1 | 1 | 1 | 0 |

## Full Adder

$S = A \oplus B \oplus C$

$C_{out} = A \cdot B + A \cdot C + B \cdot C$




Anti-Symmetry

| A | B | C | $C_{out}$ | S |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 0 | 1 | 0 | 1 |
| 0 | 1 | 0 | 0 | 1 |
| 0 | 1 | 1 | 1 | 0 |
| 1 | 0 | 0 | 0 | 1 |
| 1 | 0 | 1 | 1 | 0 |
| 1 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 1 | 1 |



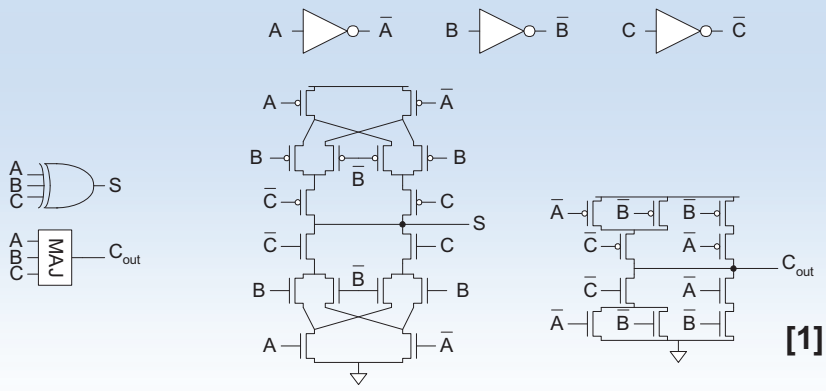


# Full Adder Design I

- direct implementation from equations

$$S = A \oplus B \oplus C$$

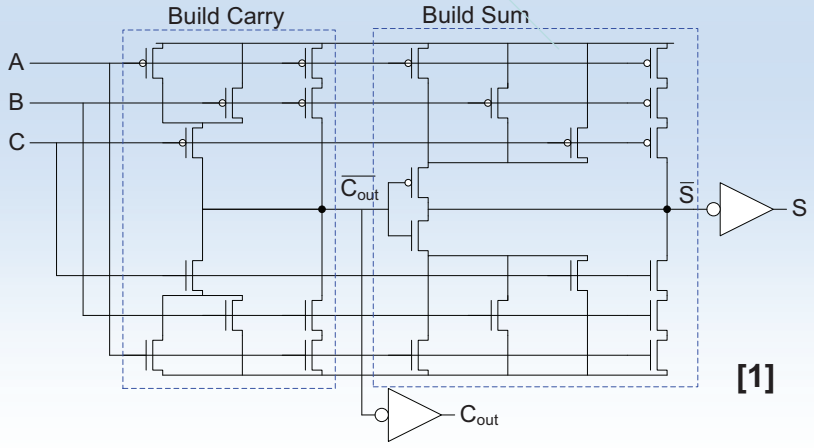
$$C_{\text{out}} = MAJ(A, B, C)$$




[1]

# Full Adder Design II

- Sum S as function of $C_{out}$
  $S = ABC + (A + B + C)(\overline{C_{out}})$
- Notice: Pull-up path fully symmetric to pull-down path because of truth table anti-symmetry




[1]





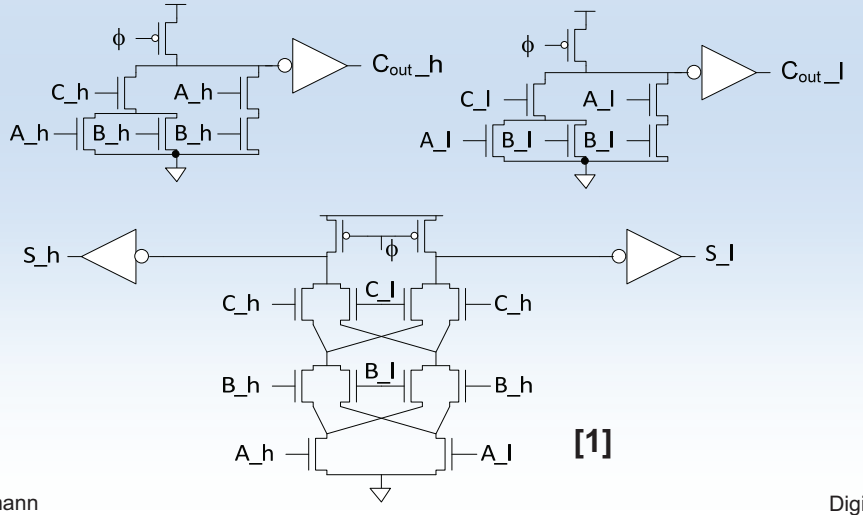
# Full Adder Design II

## Characteristics

- Layout is uniform
- Transistor count is 24+2*INV only
- Carry-bit is faster than Sum: well suited for carry-ripple adders

# Full Adder Design III

- Dual-rail domino
  - Very fast, but large and power hungry
  - Used in very fast multipliers



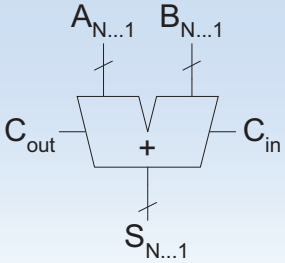

[1]

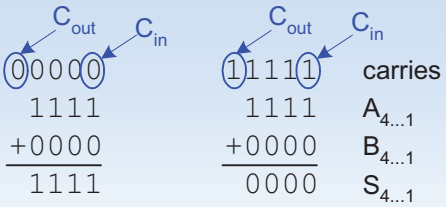




# N-bit Adders

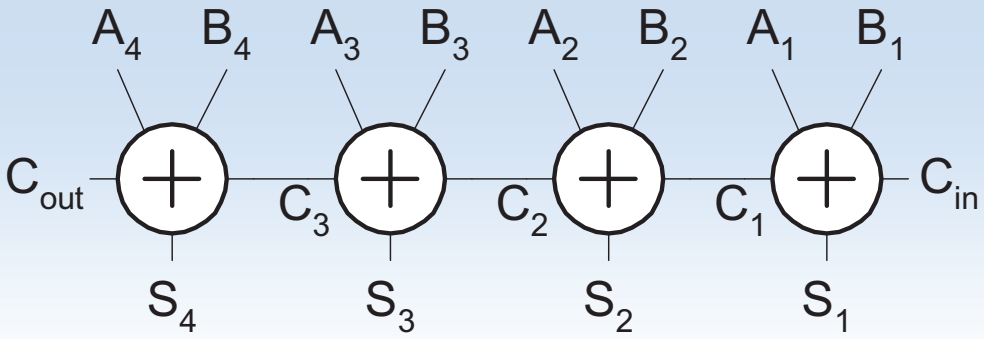
- Add two N-bit binary numbers
- N-bit adder also called Carry Propagate Adder
  - Each sum bit depends on all previous carries



# Carry-Ripple Adder

- Simplest design: cascade full adders
  - Critical path goes from Cin to Cout
  - Design full adder to have fast carry delay








# Carry-Ripple Adder

- Propagation delay:

$$t_{carry-ripple} = (N-1) \cdot t_c + t_s$$

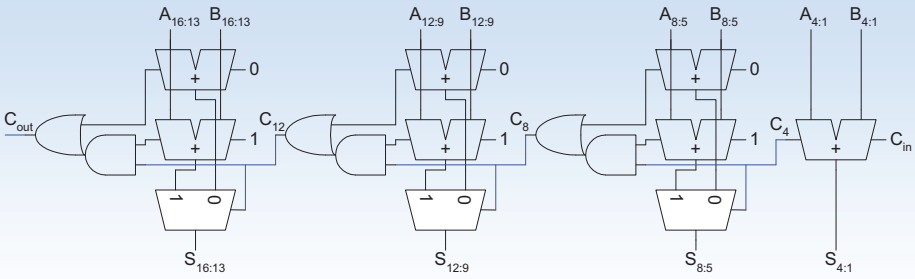
$t_c$ $carry-delay\ of\ full\ adder$

$t_s$ $sum-delay\ of\ full\ adder$

Conclusion:
Design full adder with small carry-delay

# Carry-Select Adder

- For long word length carry-ripple adder is too slow
- Idea:
  - ➢ divide N-bit adder into K blocks
  - ➢ Each block needs <u>two</u> N/K-bit adders
  - ➢ each block precomputes the sum for both: $C_{in}=0$, $C_{in}=1$

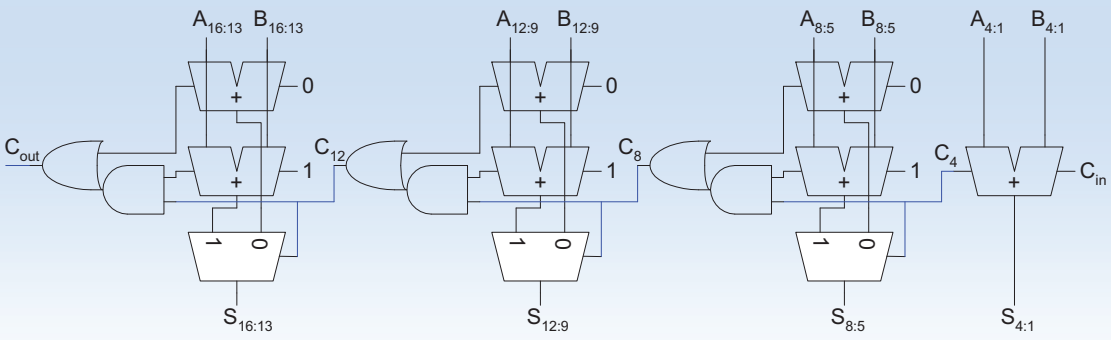




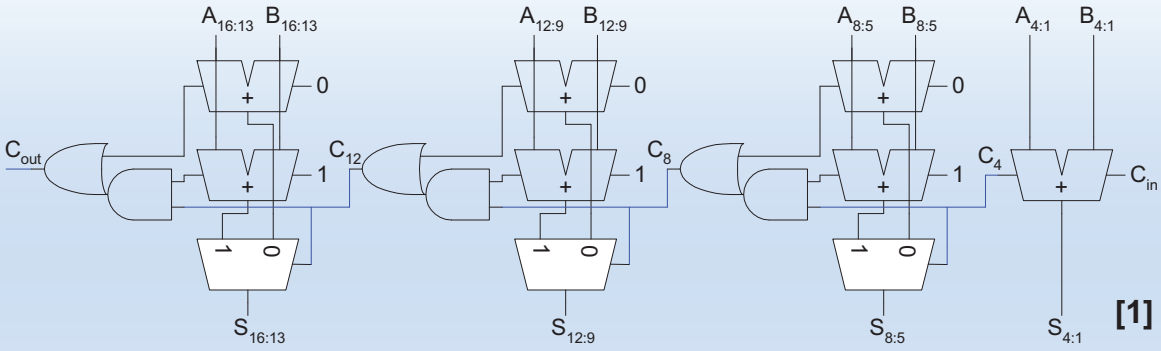



# Carry-Select Adder

- ➢ When the precomputation is ready, the carry from the previous block is also ready, deciding which sum is correct
- ➢ The correct carry-out of each block is getting calculated by an AND+OR-Gate!



# Carry-Select Adder




[1]

- Propagation delay for N-bit addition:

$$t_{carry-select} \approx t_{N/K-bit\_Adder} + (K-1) \cdot t_{AND+OR}$$

$$K \quad number\ of\ blocks$$





# Carry-Lookahead Adder

- Carry-lookahead adder computes the carry-bit for each bit position out of the input signals $A_i$,$B_i$,$C_0$, i=0….N-1
- Using intermediate signals: G(generate),P(propagate)

$$G_i = A_i \cdot B_i$$

$$P_i = A_i \oplus B_i$$

- It hold for each bit position

$$S = A \oplus B \oplus C_{in} \qquad S = P \oplus C_{in}$$

$$C_{out} = A \cdot B + C_{in} \cdot (A + B) \qquad C_{out} = G + C_{in} \cdot P$$

# Carry-Lookahead Adder

1. step: Calculate bitwise Propagate und Generate

Bitwise PG Logic

$$P_0 = A_0 \oplus B_0 \qquad G_0 = A_0 \cdot B_0$$

$$P_1 = A_1 \oplus B_1 \qquad G_1 = A_1 \cdot B_1$$

2. step: Calculate bitwise Carry

Carry Logic

$$C_1 = G_0 + C_0 \cdot P_0$$

$$C_2 = G_1 + C_1 \cdot P_1 = G_1 + (G_0 + C_0 \cdot P_0) \cdot P_1 = G_1 + G_0 \cdot P_1 + C_0 \cdot P_0 \cdot P_1$$

$$C_3 = G_2 + G_1 \cdot P_2 + G_0 \cdot P_1 \cdot P_2 + C_0 \cdot P_0 \cdot P_1 \cdot P_2$$





# Carry-Lookahead Adder

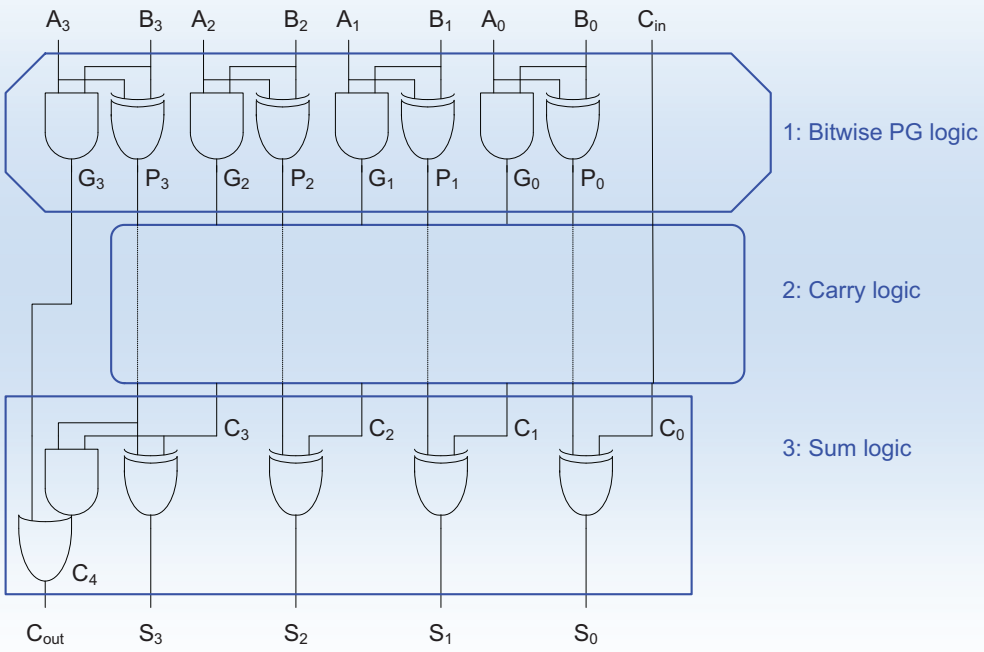
3. step: Calculate bitwise Sum

Sum Logic

$$S_0 = P_0 \oplus C_0$$

$$S_1 = P_1 \oplus C_1$$

# Carry-Lookahead Adder

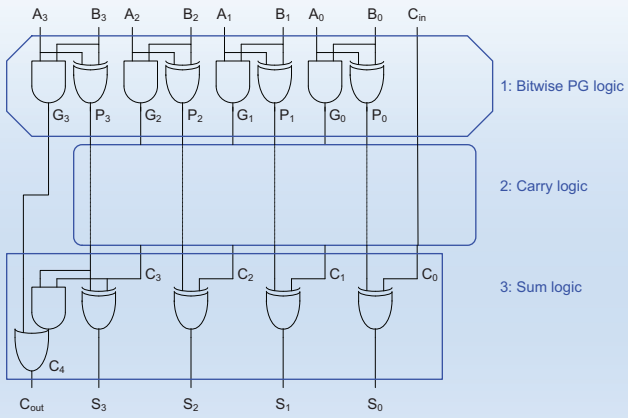



Task: Draw circuit of carry logic





# Carry-Lookahead Adder




- Propagation delay for N-bit addition:

$$t_{carry-lookahead} \approx t_{EXOR} + t_{AND} + t_{OR} + t_{EXOR}$$

Propagation delay is almost independend of N (N≤32)!

# Carry-Lookahead Adder

- But for cryptography N=512 or N=2048
- Carry logic is using N-input AND + N-input OR
- Gate delay of N-input AND/OR is function of N!

$$t_{carry-lookahead} \propto \log N$$

Optimistic approach, because distances signals have to travel on chip increase in proportion to N!





# Tree Adder

- If lookahead is good, lookahead across lookahead!
- Many variations on tree adders
  - Brent-Kung
  - Sklansky
  - Kogge-Stone
  - etc.

Adder architectures offer area / power / delay tradeoffs.

Choose the best one for your application!

# Multiplication

- Essential for μP, DSP and graphic engines
- Example:

multiplicand multiplier

$1100 \times 0101 : 12_{10} \times 5_{10}$

```
  1100
 0000
1100
0000
--------
00111100 : 60₁₀
```

partial products

product

- M x N-bit multiplication
  - Produce N partial products (N rows), each have bit with M
  - Shift and sum these partial products to produce M+N-bit product

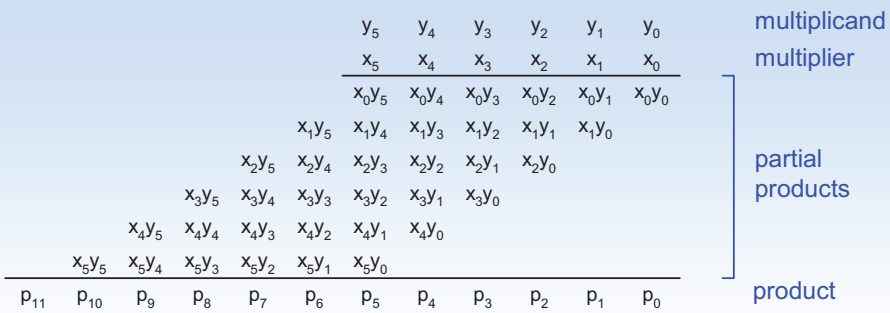




# General Form

- Multiplicand: $Y = (y_{M-1}, y_{M-2}, \ldots, y_1, y_0)$
- Multiplier: $X = (x_{N-1}, x_{N-2}, \ldots, x_1, x_0)$
- Product:

$$P = \left(\sum_{j=0}^{M-1} y_j 2^j\right)\left(\sum_{i=0}^{N-1} x_i 2^i\right) = \sum_{i=0}^{N-1}\sum_{j=0}^{M-1} x_i y_j 2^{i+j}$$



# Summing up the Partial Products

Two possible approaches of hardware implementation:

1. Add the first two rows of partial products by using a carry-propagate adder CPA (ripple-carry/carry-select/carry-lookahead). Use another CPA to add the third row of partial products and so forth. So we need N-1 adder for M x N-bit multiplication.





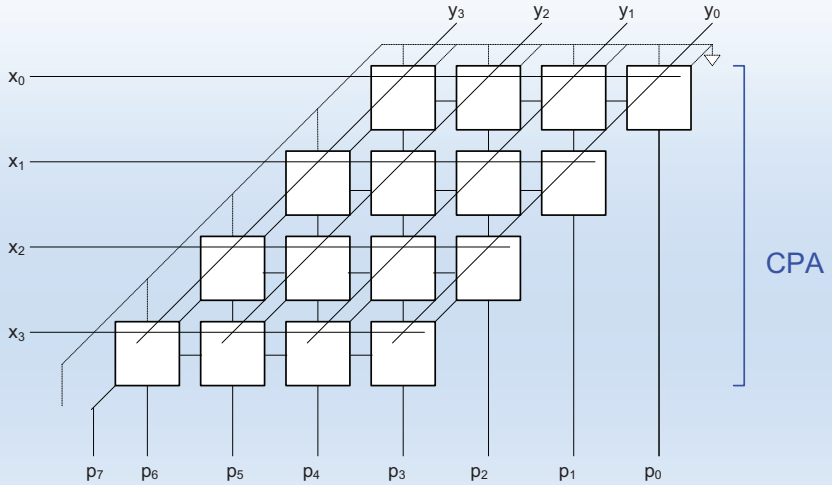
# Summing up the Partial Products

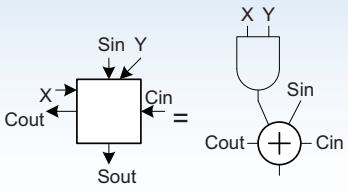
Two possible approaches of hardware implementation:

2. Add the partial product to the result of the previous row using the carry-save redundant form! → use carry-save adders (CSA)

   Finally convert the redundant form into a regular binary form! → use a carry-propagate adder (CPA)!

# Approach 1: Array Multiplier with CPA



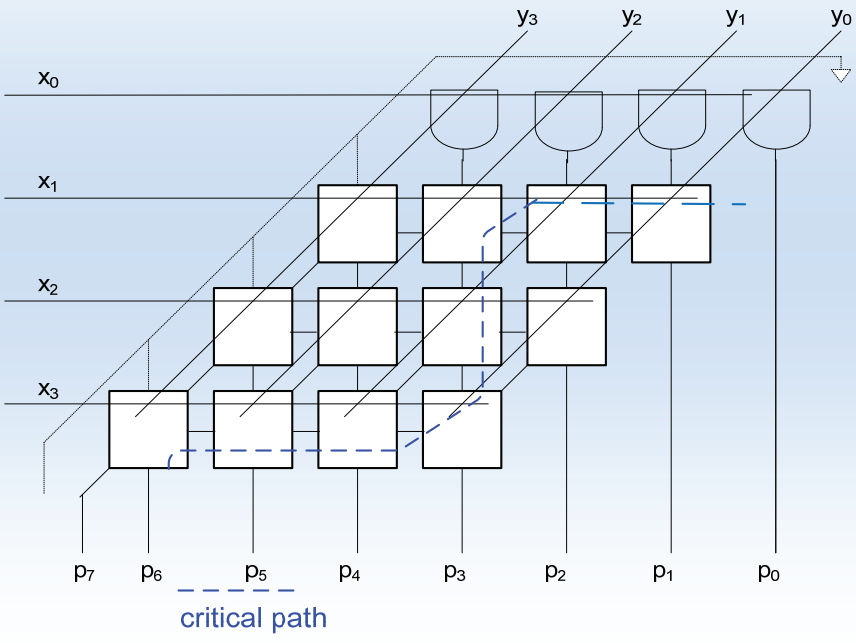







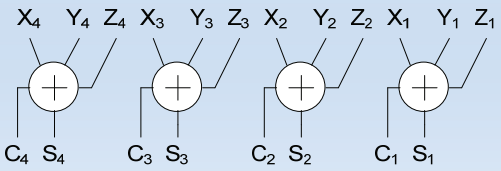

# Approach 1: Array Multiplier with CPA

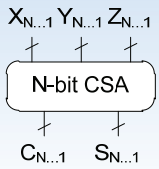
First row can be simplified using AND-Gate only



# Carry Save Addition (CSA)

- A full adder sums 3 inputs and produces 2 outputs
- N full adders in parallel are called *carry save adder*
  - Produce N sums and N carry outs called the carry save redundant form output




Each N-bit CSA adds 3 words of N bit

$$t_{CSA} = \max(t_c, t_s)$$

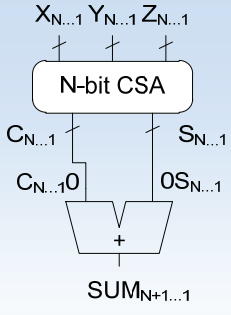




# Carry Save Addition (CSA)

Final sum can be calculated by:

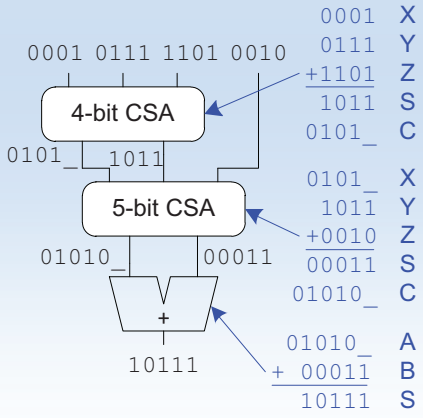
1. Shifting the carry left by one place
2. Appending a 0 as most significant bit of the sums
3. Using a carry-ripple adder to add sums and carry outs together to produce N+1 bit final binary sum



# CSA Application

- Multi input adder: K words
  - Use K-2 CSA
  - Use final CPA to compute actual result

Example: K=4




[1]

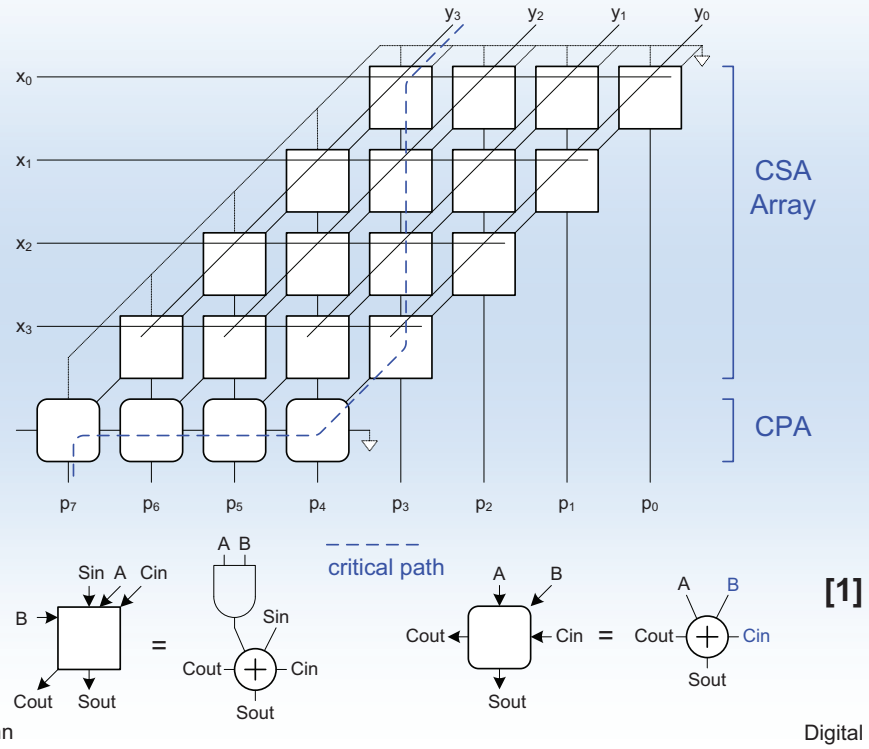




# CSA Application

- M x N-bit Multiplication
  - Use N stages of CSAs (M full adder + AND-Gate)
  - Keep result in carry-save redundant form
  - Use final CPA to compute final result in binary form

# Approach 2: Array Multiplier with CSA



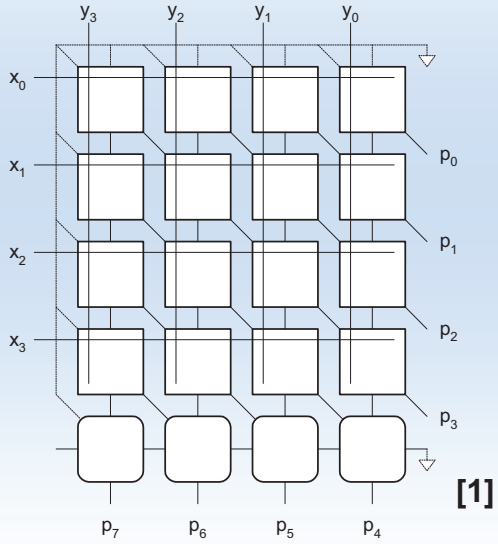

[1]





# Rectangular Array

- Squash array to fit rectangular floorplan




[1]

# Array Multiplier: Summary

- Both implementation approaches shows approx. same propagation delay (latency)!
- Notice that within the CSA array the full adder should have equally fast carry and sum propagation delay, because the greater of both limits the speed performance!
- Data flow within the CSA array is vertically only! So the array multiplier with CSA can be easily pipelined with the placement of registers between rows!





# Advanced Multipliers

- Number of partial products can be reduced using a redundant number representation e.g. Booth-multiplier
- Try to transform serial operations into parallel operations to speed up the process of multiplication e.g. Wallace-tree
- Add sign extension: partial product can be negative